第九章　磁盘存储器的管理

主要内容

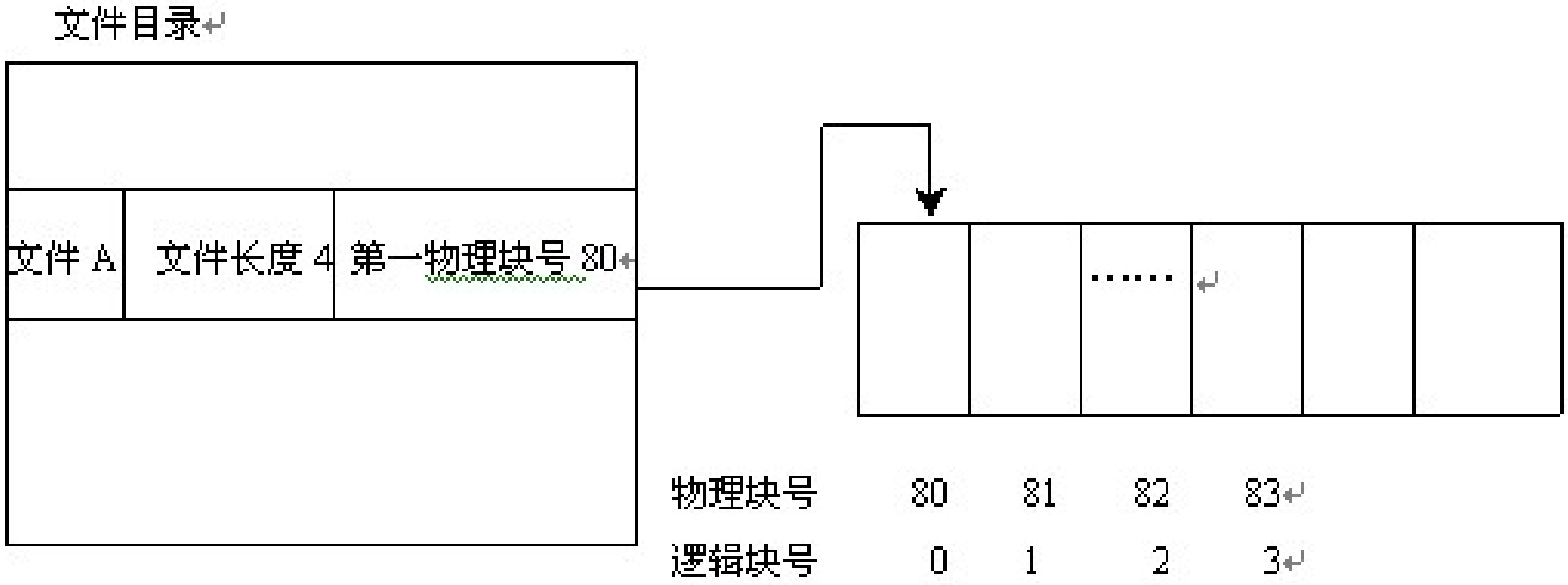
- 文件的物理结构（外存的组织方式）
- 文件存储空间的管理

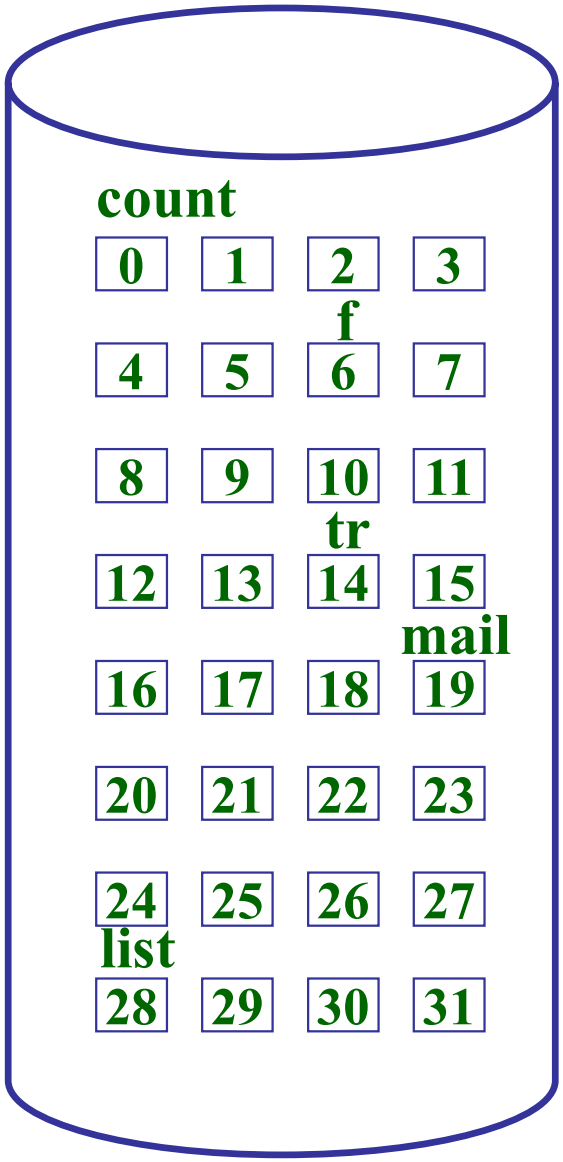
9.1 外存的组织方式

- 即文件的物理结构，又称文件的存储结构，是指文件在外存上的存储组织形式。
- 两个问题：
  - 有效利用外存空间
  - 提高文件访问速度
- 三种结构：
- 1、连续分配/结构
- 2、链接分配/结构
- 3、索引分配/结构

1、连续结构

- 每个文件分配一组相邻盘块。





文件目录

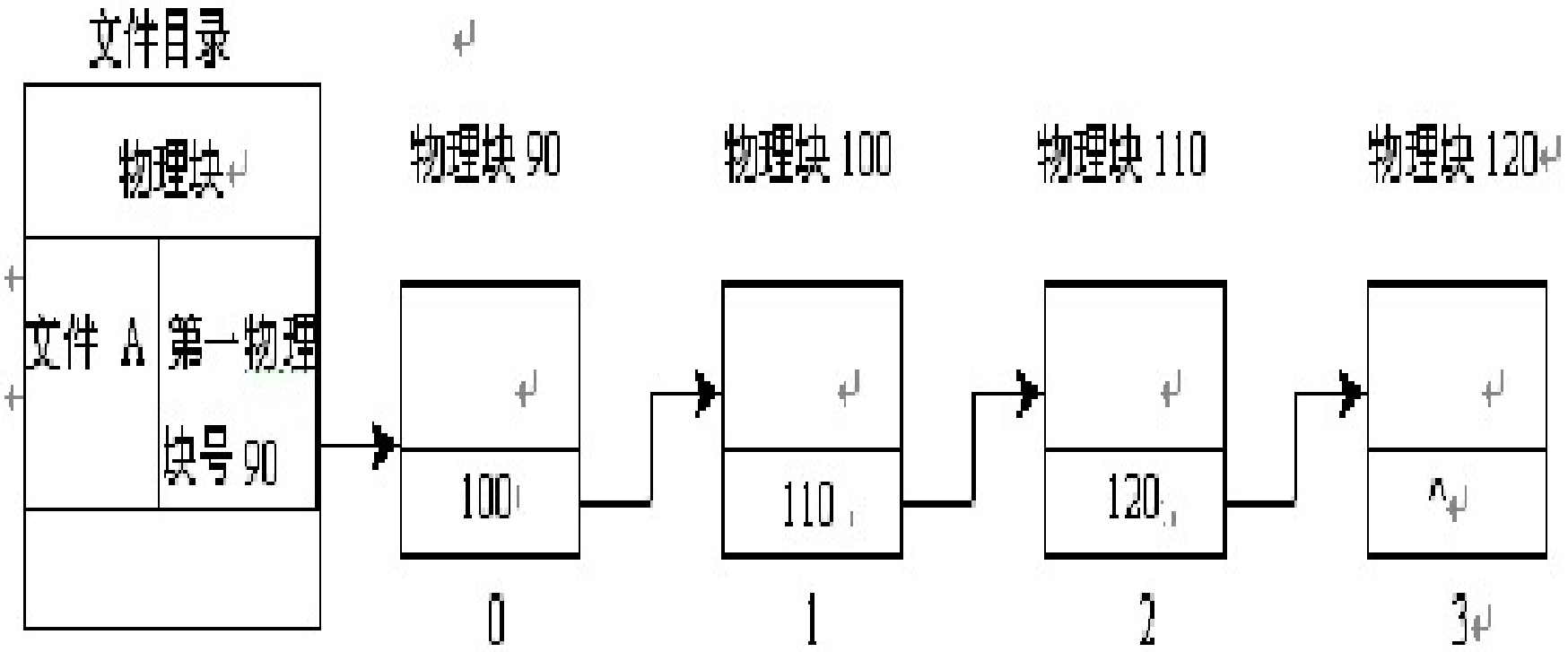
文件名	始址	块数
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

优缺点

- 优点：
  - 顺序访问容易
  - 顺序访问速度快（磁头移动距离小）
- 缺点：
  - （1）要求连续空间
  - （2）必须事先知道长度，文件不易动态增长和删除。
    - 预留空间:浪费
    - 重新分配和移动

2、链接结构

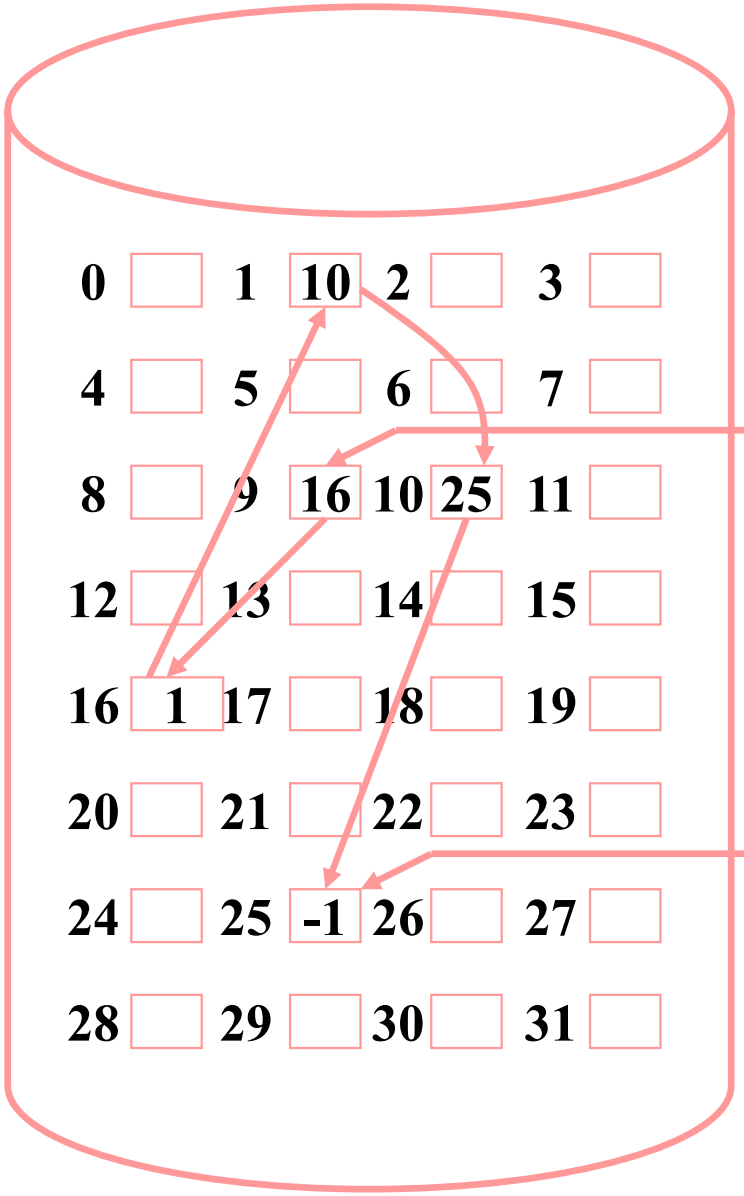
- 是一种<u>非连续</u>的结构，文件离散地分配于各盘块中，将同属于同一个文件的多个离散的盘块连接成一个链表。
- 分类：
  - 隐式链接
  - 显式链接



对应目录项：链表的首指针

文件目录

文件名	始址	末址
jeep	9	25



例题：

- 链接文件由**5**个逻辑记录组成，每个逻辑记录大小等于磁盘块大小，为**512**字节，依次放在**50**，**121**，**75**，**80**，**63**号物理块上
- 问：**1569**逻辑字节在哪个物理块上？

解答

- 因为：**1569=512*3+33**
- 所以：**1569**逻辑字节在逻辑记录 **3** 上
- 逻辑记录 **3**对应 **80**号物理块

优缺点

- 优点：
  - 提高了磁盘空间利用率,不存在外部碎片问题
  - 有利于文件插入和删除，动态扩充
- 缺点：
  - 存取速度慢，不适于随机存取
  - 链接指针占用一定的空间
  - 可靠性问题，如指针出错

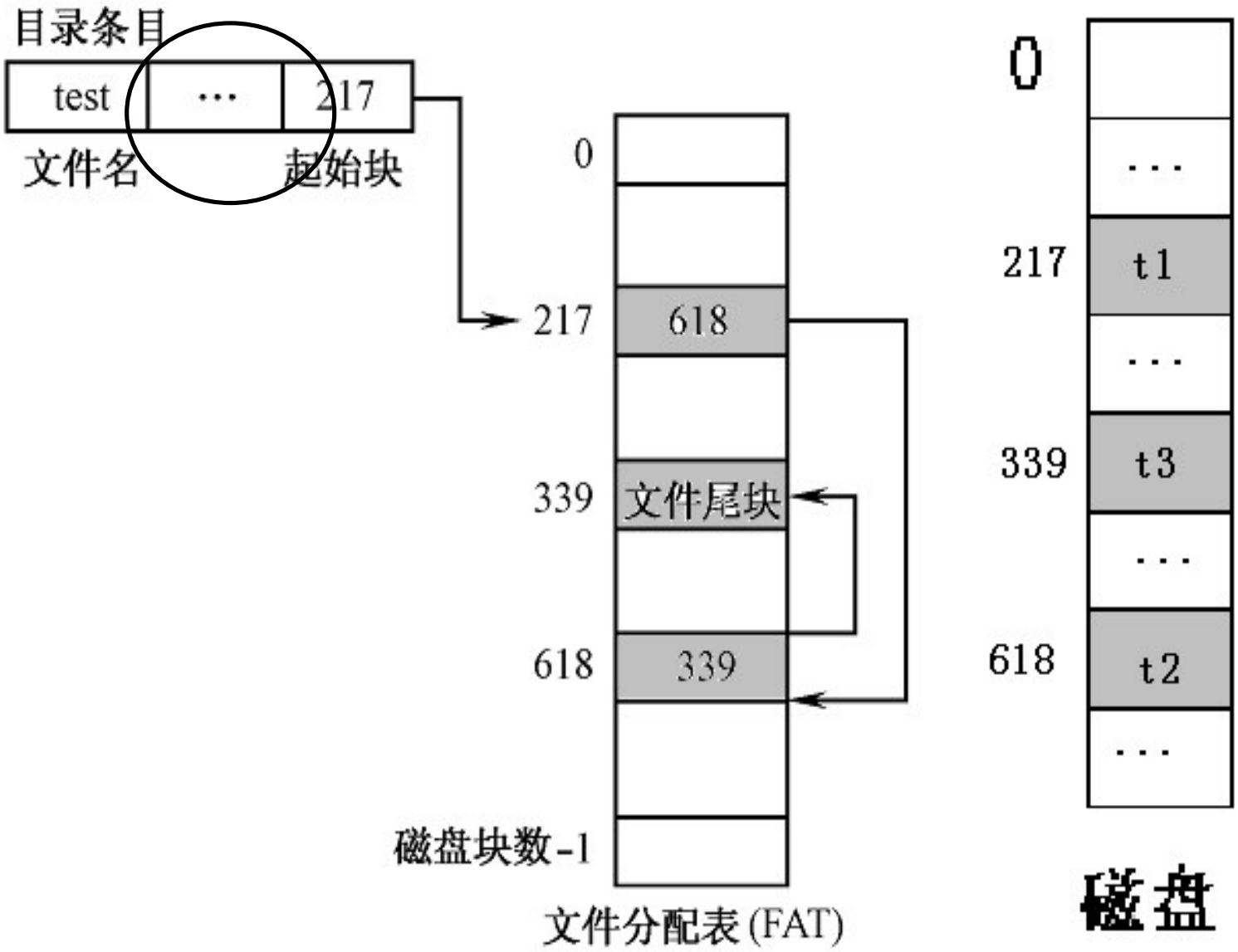
隐式链接

- 文件目录表中有第一个、最后一个块号，每个盘块中有下一块号。
- 特点：只适合于顺序访问，对随机访问效率低，可靠性差。
- 簇：包含多个块的单位，当以它为单位分配并链接，可减少访问时间，但增大了内部碎片尺寸

显式链接

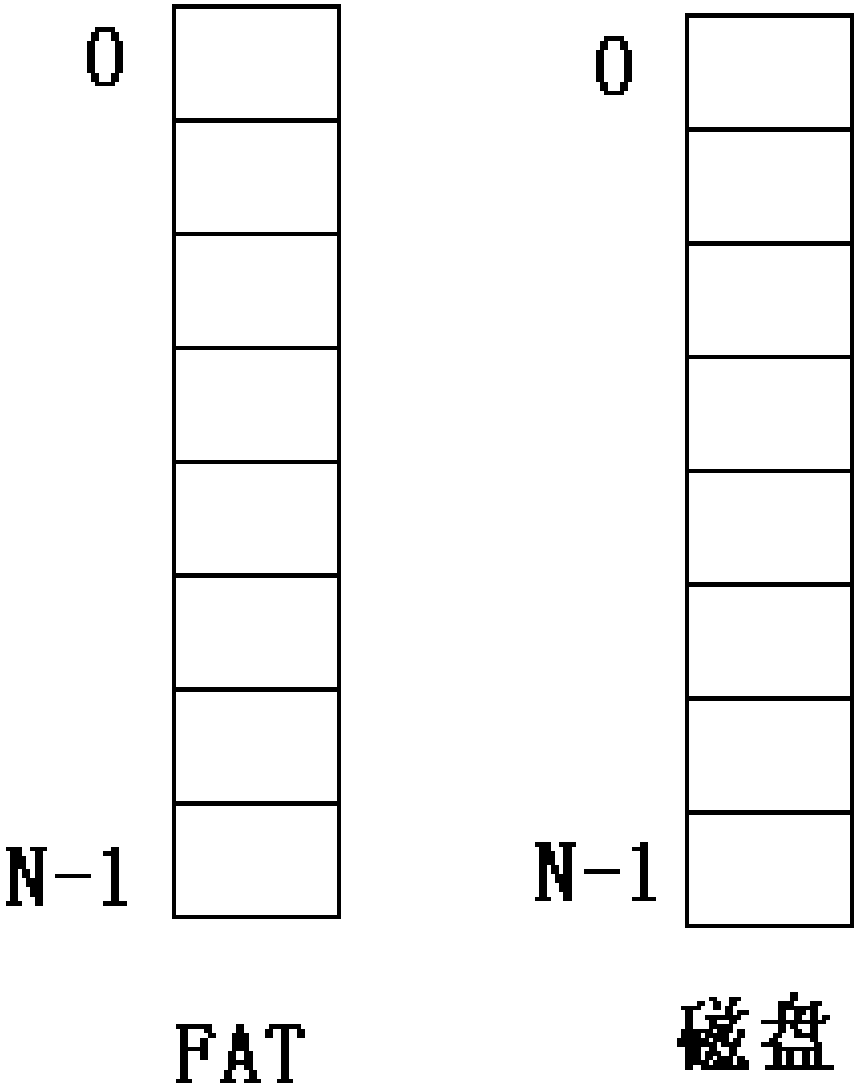
- 把用于链接各物理块的指针显式存放在一张表（整个磁盘一张，称文件分配表FAT ）中，该表系统启动后存储在内存中。
- 利用FAT可方便地进行随机存取。
- 但FAT要占用一定的存储空间，若盘的容量较大时要占用较大的存储空间，甚至可能在内存中装不下整个FAT。

图示


目录条目
test
…
217
文件名
起始块
0
217
618
339
文件尾块
618
339
磁盘块数-1
文件分配表(FAT)
0
…
217
t1
…
339
t3
…
618
t2
…
磁盘

FAT的实例

- 在**MS-DOS**和**Windows**系统中，文件的物理结构使用的是**FAT**结构。
- 将磁盘空间划分为块（称为簇），每块大小为扇区的整数倍。
- 一个磁盘分区分为多少簇则**FAT**就有多少表项
- 例如：
  - 对于1.2MB软盘，盘块大小为1KB
  - 共1.2M/1K=1.2k个磁盘块，每个FAT表项占12bit=1.5Bytes，
  - FAT空间=每个FAT表项占1.5B*共1.2k个表项，故共1.8kB.


0
N-1
FAT
0
N-1
磁盘

习题：

- 若磁盘块大小为**1K**，对**540M**，**1.2G**两种容量的磁盘
- 问：**FAT**表分别需要多少空间？

解答

- 盘块数为 **540M/1K=540K**个
  - 因为**512K<540k<1024k**
  - 所以为了表达盘块地址需**20bit**即**2.5 byte**
  - 所以**FAT**占空间**2.5B*540k=1350kB**
- 同理：
  - **1.2G**磁盘的盘块数为 **1.2G/1K=1.2M**个
  - **1M<1.2M<2M , 2M**需要**21bit** ，
  - 若近似到半字节， **->24bit 3Bytes**
  - **3B*1.2M=3.6MB**

- **FAT12:**
  - **4096**
  - 簇
- **FAT16**
  - **65536**
- **FAT32**
  - 簇：固定**8**个盘块，**4K**
  - **2TB**

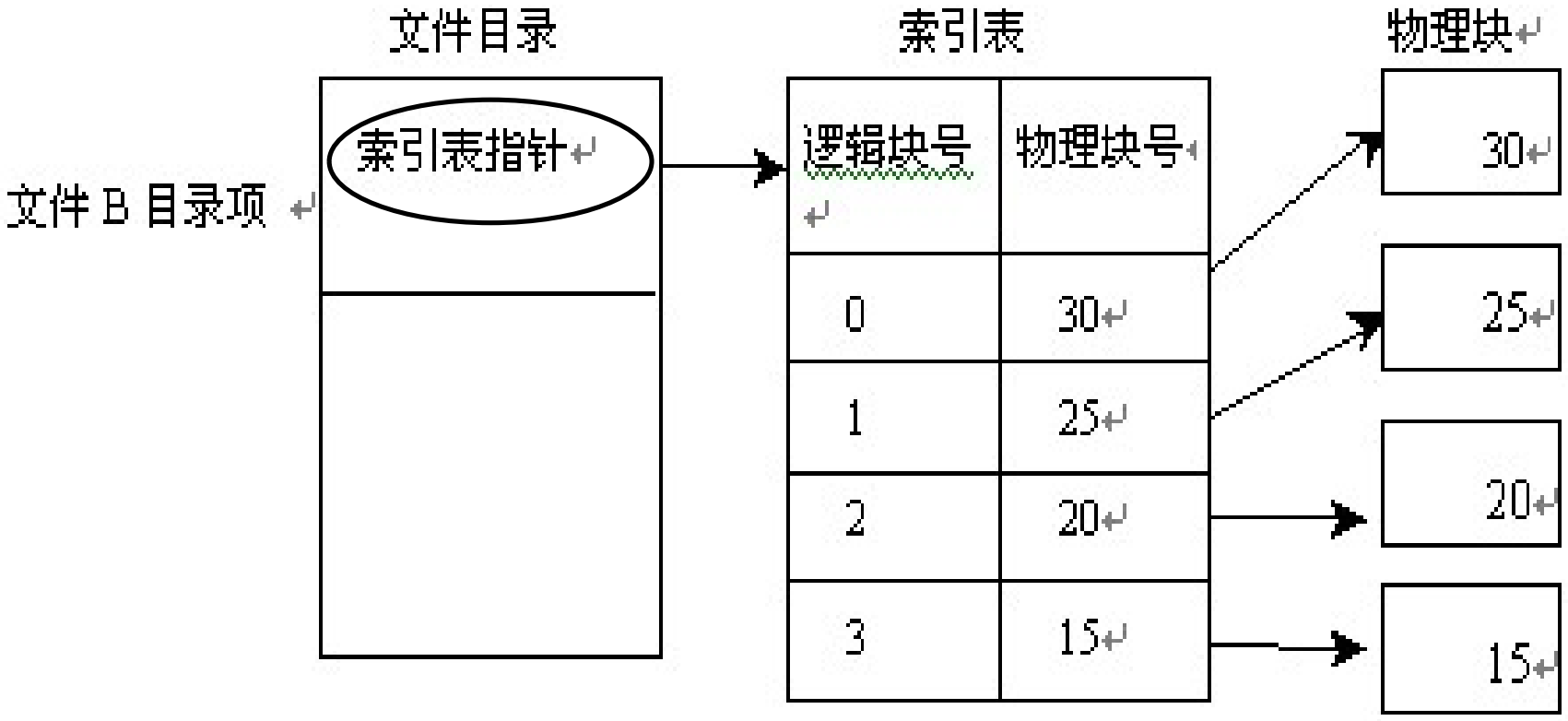
NTFS(*)

- **NTFS(New Technology File System)**是专为**Windows NT**设计的一种具有较好容错性和安全性的文件系统，是第一个为高端服务器和工作站提供健全的文件服务的文件系统。
- **64**位盘地址
- 长文件名：单名**255**，路径名**32767**
- 系统容错
- 数据一致性
- 文件加密压缩
- 分配：簇：簇的大小（**512K-64K**，默认**4K**），“卷因子”，格式化

3、索引结构

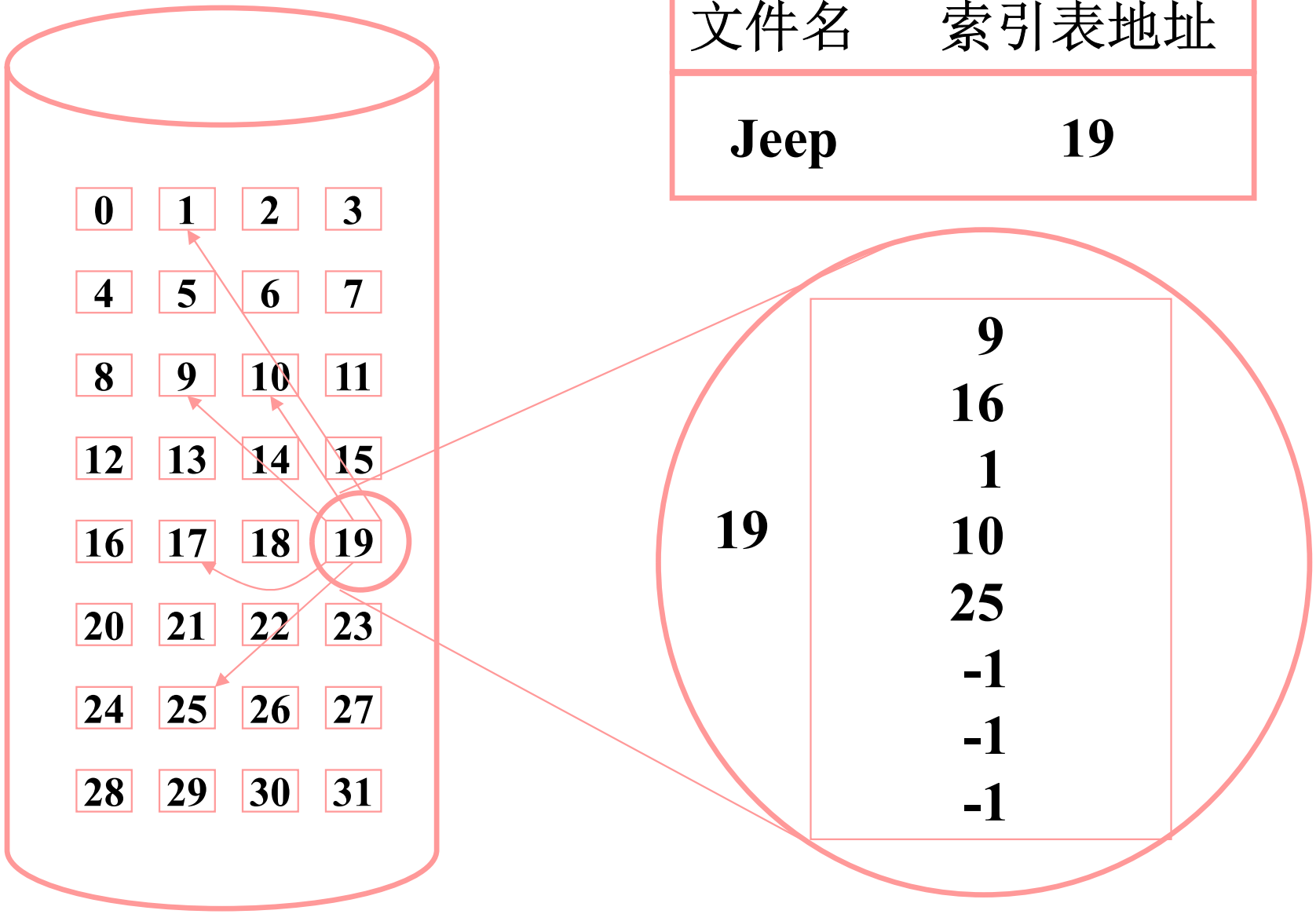
- 一个文件的信息存放在若干不连续物理块中，系统为每个文件建立一个专用数据结构--索引表，保存这些物理块的块号，其中第i个条目指向文件的第i块
- 一级索引分配
- 多级索引分配
- 混合索引分配

一级索引分配



文件目录

文件名	索引表地址
Jeep	19



优缺点

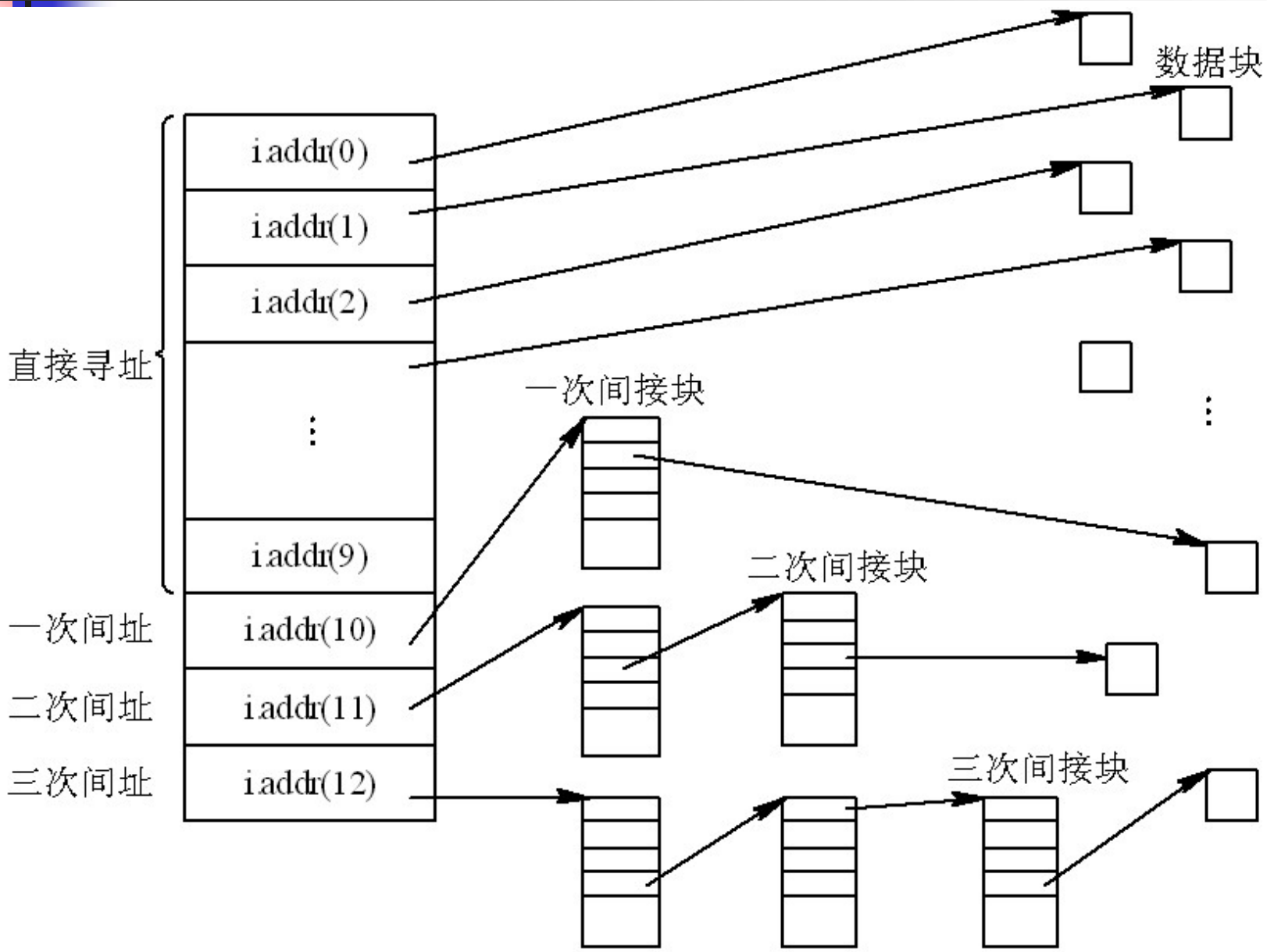
- 优点
  - 保持了链接结构的优点，又弥补了其不足
  - 既能顺序存取，又能随机存取
  - 满足了文件动态增长、插入删除的要求
  - 能充分利用外存空间，不产生外部碎片
- 缺点
  - 索引块本身带来了系统开销
  - 小文件的索引块利用率低

多级索引分配

- 当大文件的索引块太多时，可以：
  - 通过链接指针将各索引块按序链接起来；
  - 为这些索引块再建立一级索引，
    - 如果文件非常大时，还可用三级、四级索引分配方式。

混合索引分配方式

- 即将多种索引分配方式结合。
- **UNIX**系统中每个文件都有一个索引结点，称**I**结点**(I-node)**
  - 其中有**13**个指针用于物理空间分配。
    - 前**10**个指针指向可以直接访问的磁盘块。
    - 剩下**3**个指针指向间接块，即不包含数据的块，分别指向：一级间接块、二级间接块、三级间接块。


数据块
iaddr(0)
iaddr(1)
iaddr(2)
直接寻址
一次间接块
iaddr(9)
二次间接块
一次间址
iaddr(10)
二次间址
iaddr(11)
三次间址
iaddr(12)
三次间接块

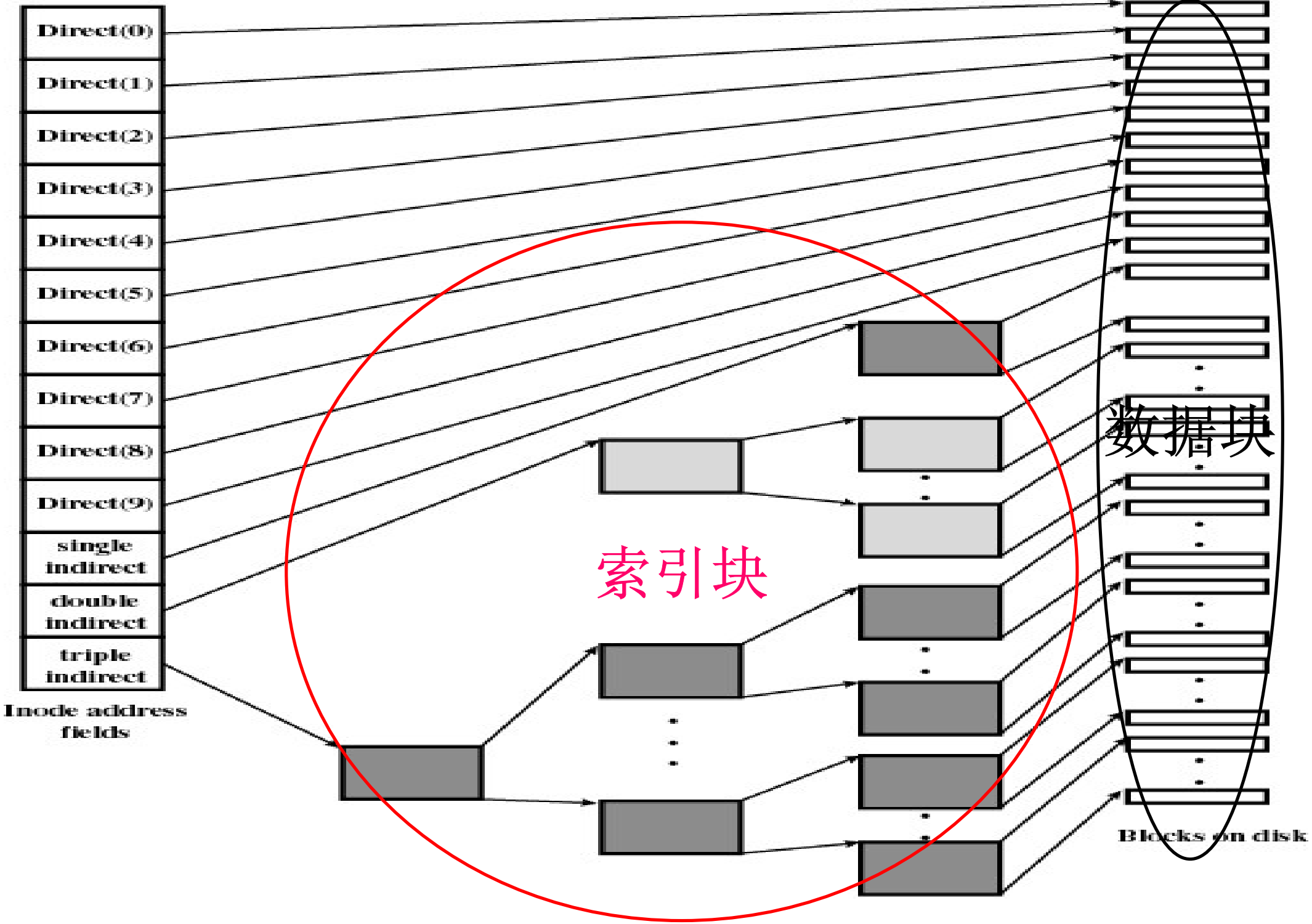


Figure 12.13 UNIX Block Addressing Scheme

- 假设盘块大小为**4KB**，盘块号长度为**4**字节
- 则一个盘块可存储**1K**个盘块号
- （**1**）文件不大于**10x4KB=40KB**时，可从**I**结点直接得到文件的全部盘块号。
- **(2)**一次间址块可放**1K**个盘块号，可满足文件长度不大于**40KB+4MB**
- **(3)**二次间址块，**40KB+4MB+4GB**
- **(4)**三次间址块，**40KB+4MB+4GB+4TB**

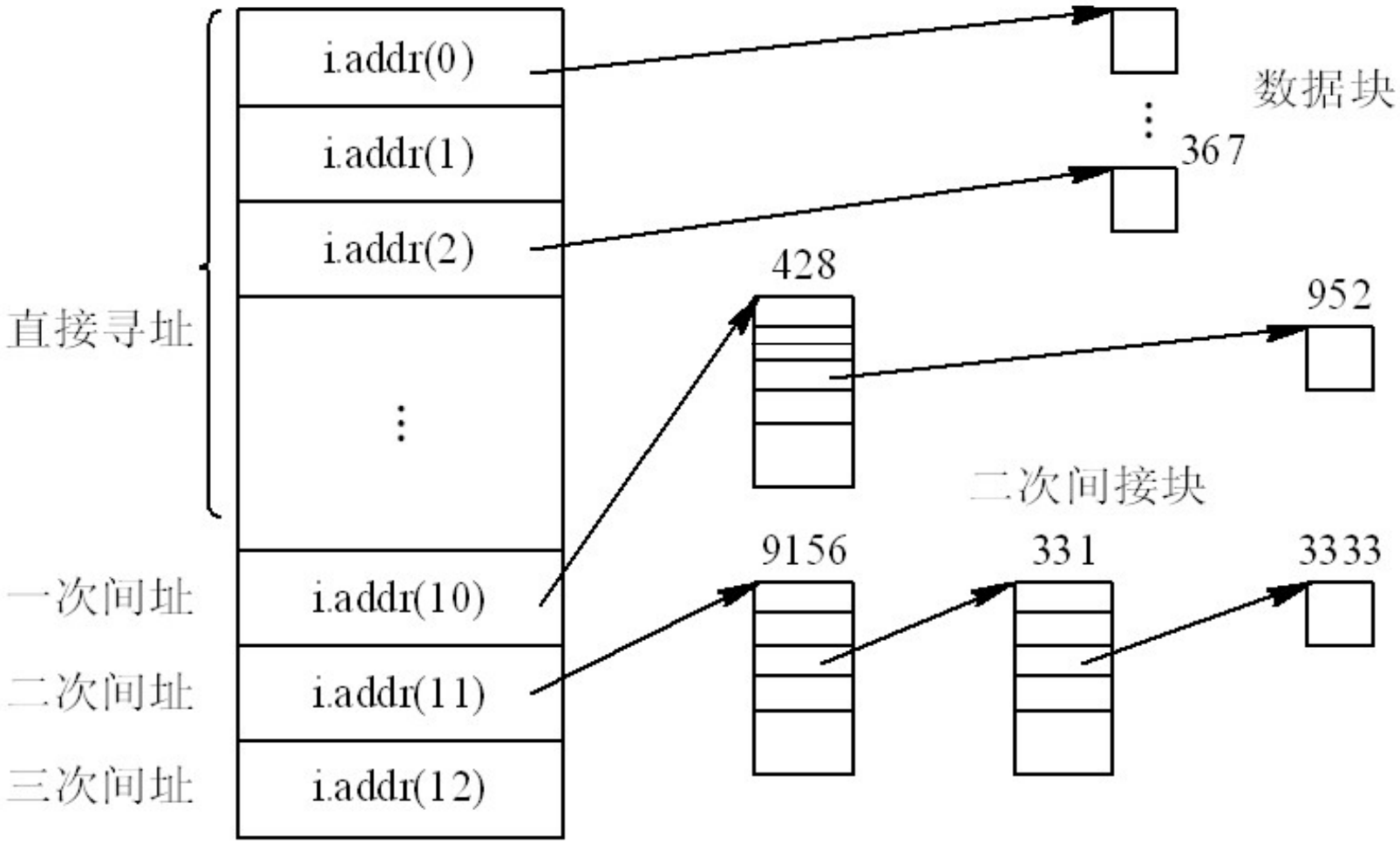
例题： 地址转换

1）将逻辑文件字节偏移量转换为文件逻辑块号

2）把文件逻辑块号转换为物理盘块号

假设：盘块大小1k 盘块号占32bit（4Byte）

文件地址映射示例



逻辑地址3000=2K+952　物理地址 367k+952

逻辑14000=13K+688　物理952k+688

9.2文件存储空间的管理

- 策略：连续、离散
- 分配单位：块
  - 几乎所有的文件系统都把文件分成固定大小的块来存储，扇区、磁道和柱面都可以作为分配单位；
  - 块太大，会浪费太多空间；
  - 块太小，读取速度变慢。
  - 常见块大小：**512B**，**1KB**或**2KB**
- 分配与回收
- 空闲空间管理

空闲块管理

- 1. 空闲块表（空白文件目录）
- 将所有空闲块记录在一个表中，即空闲块表
- 2. 空闲块链表
- 把所有空闲块链成一个链
- 3. 位图法
- 用一串二进制位反映磁盘空间中分配使用情况，每个物理块对应一位
- **4.UNIX的成组链接法**

1、空闲块表

序号	第一空白块号	空白块个数	物理块号
1	**2**	**4**	**（2，3，4，5）**
2	**9**	**3**	**（9，10，11）**
3	**15**	**5**	**（15，16，17，18，19）**
4	**—**	**—**	**—**

- 每个空闲区一个表目。
- 表目包括：第一空白物理块块号、空白块的数目。
- 请求分配存储空间时，系统依次扫描各表目，直到找到一个合适的空闲区为止。

- 回收：
  - 扫描空闲表，寻找一个空表目，并将释放空间的第一物理号及它所占的物理块数填到这个表目中。（注意合并）
- 特点：
  - 仅当有少量的空白区时才有较好的效果
  - 如果有大量的小的空白区，则目录变得很大，效率大为降低。
  - <u>适用于建立连续文件</u>。

2. 空闲块链

- 空闲盘块链：以盘块为单位拉成链
  - 把所有的“空白块”链在一起。
  - 创建文件需要一个或几个物理块时，就从链头依次取下一块或几块。
  - 回收文件时回收块链到空白链上。
- 空闲盘区链：以盘区为单位拉成链

3、位示图

- 用一串二进制位记录磁盘空间分配使用情况
- 每个物理块对应一位，“1”表示对应的物理块已分配，“0”表示未分配
- 申请：
  - 在位示图中查找为0的位，返回对应物理块号，置1
- 归还：
  - 将物理块号对应位置0

位示图盘块的分配

- 查找为0的位，返回对应物理块号，置1
- **(1)**顺序扫描位示图，找出一个或一组其值为“**0**”的二进制位。
- **(2)** 将所找到的一个或一组二进制位， 转换成与之相应的盘块号。
  - 位示图的第**i**行、第**j**列相应的盘块号：
  - **b=n(i-1)+j** ，其中**n**代表每行的位数。
- **(3)** 修改位示图， 令**map**［**i,j**］**=1**

位示图盘块的回收

- 将物理块号对应位置0
- **(1)** 将回收盘块的盘块号**b**转换成位示图中的行号和列号。转换公式为：

  $$i=(b-1)\text{DIV } n+1$$

  $$j=(b-1)\text{MOD } n+1$$

- **(2)** 修改位示图,令**map［i,j］=0**

- **n**个磁盘块需要**n**位的位图。
- 例：物理块**1K**，**200M**的磁盘
  - 有**200M/1K=200K**个盘块
  - 位示图需要**200k bit**
  - 即需**200k bit = 25K Bytes**，
  - 所以需要**25**个物理块存储位示图。

- 特点：描述能力强，适合各种物理结构

例题：

- 某磁盘有**10**个盘面，每个盘面**100**个磁道，**16**扇区/磁道，以扇区为分配单位：
- 问：位图方式需占多大空间？

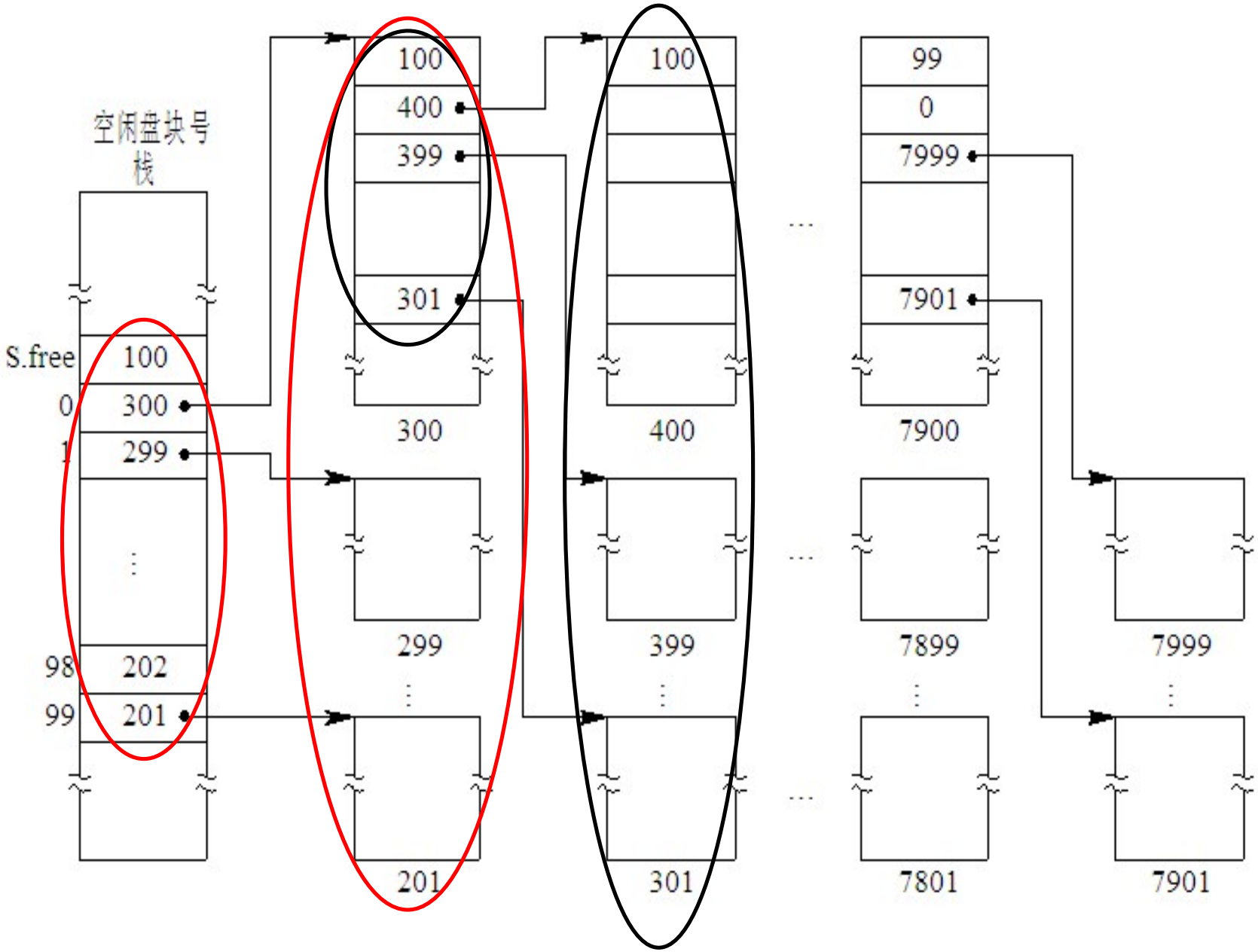
解答

- 总扇区数 **10*100*16=16000**
- 所以位图需：**16000bit=2000**字节

成组链接法

- 空闲盘块号栈：存放当前可用的一组空闲盘块的盘块号(最多为**100**个)，以及栈中尚有的空闲盘块号数**N**，**N**兼作栈顶指针用。
- 分组
  - 文件区中的所有空闲盘块被分成若干个组，如将每**100**个盘块作为一组。
- 成链
  - 由各组的第一个盘块链成一条链。
  - 将每一组含有的盘块总数**N**和该组所有的盘块号记入其前一组的第一个盘块中。

成组链接图示


空闲盘块号栈
S.free
100
0
300
1
299
98
202
99
201
100
400
399
301
300
299
201
100
400
399
301
99
0
7999
7901
7900
7899
7801
7999
7901

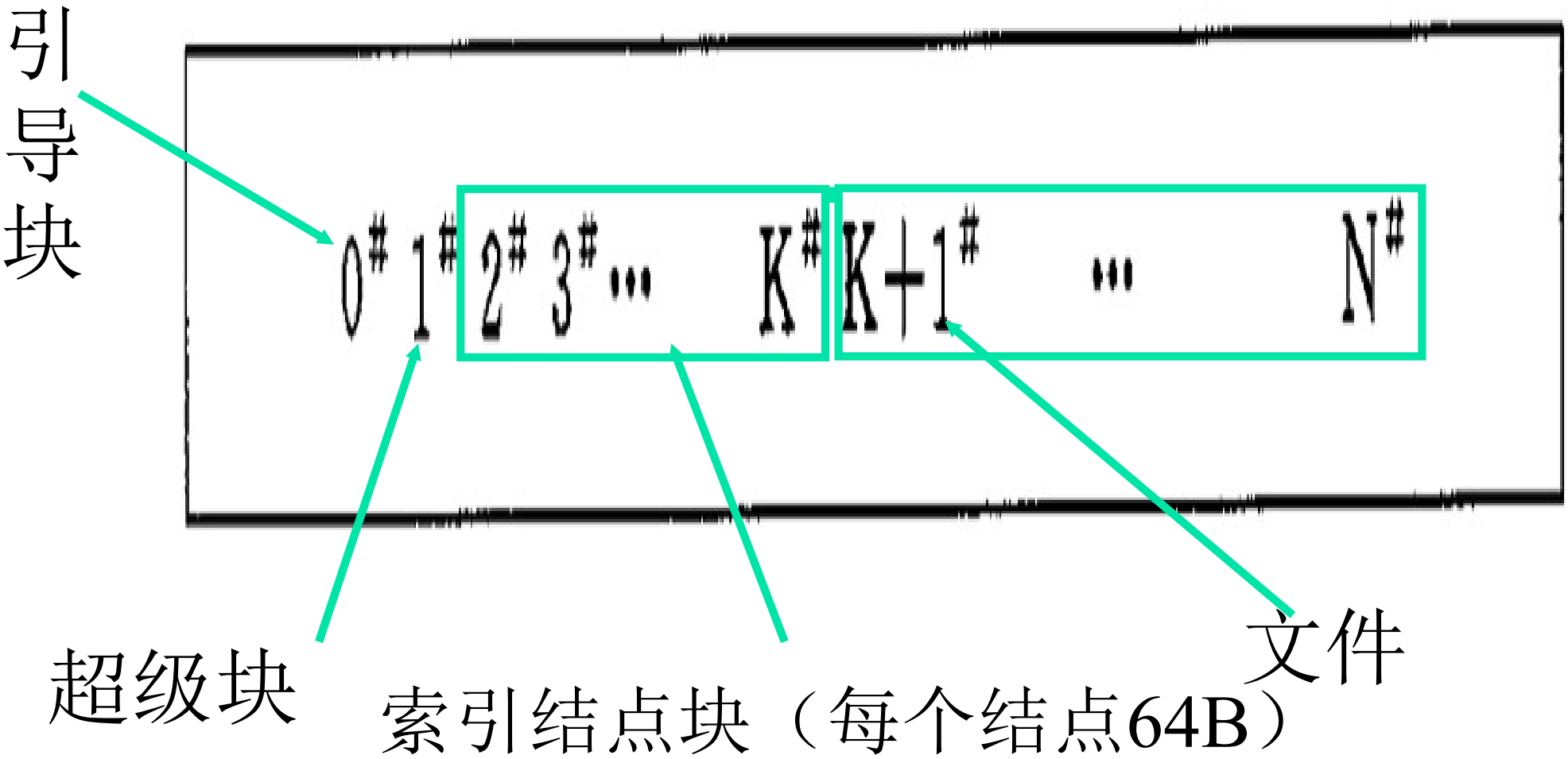
空闲盘块的分配

- 检查空闲盘块号栈是否上锁。
  - 如未上锁，从栈顶取出一空闲盘块号**A**;
  - 若盘块号**A**不是栈底，将该盘块直接分配给用户，将栈顶指针下移一格，把栈中的空闲盘块数**N**减**1**
  - 若盘块号**A**已是栈底，将盘块**A**中保存的下一组可用的盘块号读入盘块号栈中；并把盘块**A**分配出去。

空闲盘块的回收

- 若盘块号栈未满，将回收盘块的盘块号**A**，记入空闲盘块号栈的顶部，空闲盘块数加**1**。
- 当栈中空闲盘块号数目已达**100**时，即到**s.free(100)**时，将空闲盘块栈中内容放入新的回收块中，再将新回收盘块号作为新的栈底。

UNIX文件卷的组织

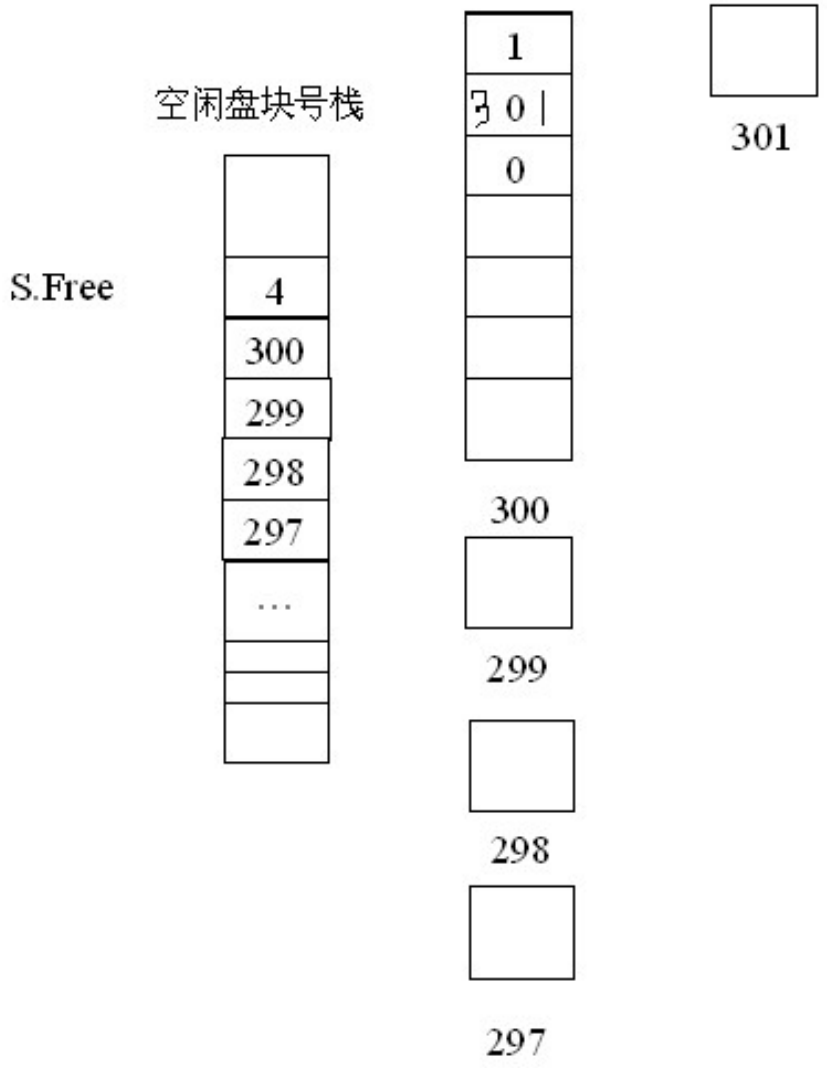


超级块(Superblock)1#块：记录文件系统中盘块和磁盘索引结点使用情况。

(1) 文件系统的盘块数目
(2) 空闲盘块号栈
(3) 当前空闲盘块号数目
(4) 空闲磁盘i结点号栈
(5) 空闲磁盘i结点数目
(6) 空闲盘块编号栈的锁字段
(7) 空闲磁盘i结点栈的锁字段
(8) 超级块修改标志
(9) 修改时间

习题：

- 成组链接法管理空闲盘块,每组**4**块,当前情况如图，请画出：
  - 回收**801**，**802**块后的情况；
  - 回收后，有一进程又需要分配**4**块空闲盘块画出分配之后的情况。



习题：

- 存放在某个磁盘上的文件系统，采用混合索引分配方式，其**FCB**中共有**13**个地址项，第**0-9**个地址项为直接地址，第**10**个地址项为一次间接地址，第**11**个地址项为二次间接地址，第**12**个地址项为三次间接地址。如果每个盘块的大小为**1024**字节，而每个盘块最多存放**250**个盘块地址：
- **(1)**该文件系统允许文件的最大长度足多少**?**
- **(2)**将文件的字节偏移量**150000**转换为物理块号和块内偏移量。
- **(3)**假设某个文件的**FCB**已在内存，但其他信息均在外存，为了访问该文件中某个位置的内容，最少需要几次访问磁盘，最多需要几次访问磁盘**?**

- **(1)**该文件系统中一个文件的最大长度可达：**10+250+250X250+250X250X250**
- **=260+62500+15625000**块**=15687760** 块
- **= 1KB* 15687760 = 15687760KB**
- **(2)550000/1024:**商为**537**，余数为**112**，即字节偏移量**550000**对应的逻辑块号为**537**，块内偏移量为**112**。
- 由于**10+250<537<10+250+250X250**，
- 而**537-(10+250)=277**，
- **277/250:**商为**1**，余数为**27**，
- 故可从**FCB**的第**11**个地址项，即二次间址项中得到二次间址块的地址，并从二次间址块的第**1**项中获得一个一次间址块的地址，再从该一次间址块的第**27**项中获得对应的物理盘块号，块内偏移量为**112**。
- **(3)**由于文件的**FCB**已在内存，为了访问文件中某个位置的内容，
- 最少需要**1**次访问磁盘**(**即可通过直接地址直接读文件盘块**)**，
- 最多需要**4**次访问磁盘**(**第一次是读三次间址块，第二次是读二次间址块，第三次是读一次间址块，第四次是读文件盘块**)**。

磁盘容错技术（略）

（1）通过存取控制机制来防止由人为因素所造成的文件不安全性。

（2）通过磁盘容错技术，来防止由磁盘部分的故障所造成的文件不安全性。

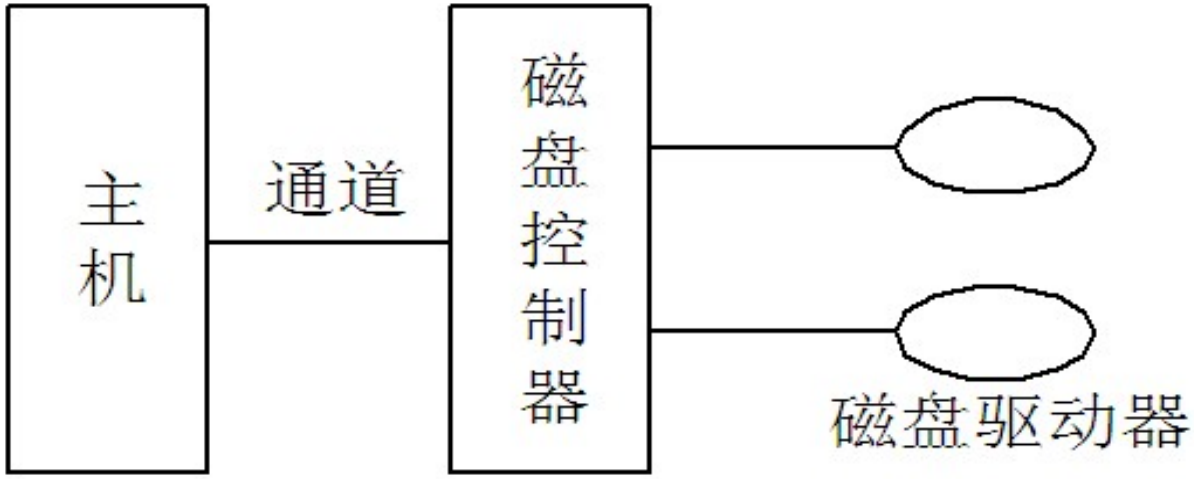
（3）通过“后备系统”来防止由自然因素所造成的不安全性。

（1）第一级容错技术SFT－I

- 低级磁盘容错，解决磁盘表面发生问题引起的数据丢失。采用双份目录，双份文件分配表及写后读校验等。系统每次启动时，对两份FAT检查是否一致。
- 热修复重定向：在磁盘中划出一部分作为热修复重定向区，存放坏磁道的待写数据
- 写后读校验：写盘后，马上从盘读出与内存校验看是否一致，不一致，重写入热修复重定向区，标记坏盘块。

（2） 第二级容错技术SFT-II

- 中级容错，针对磁盘驱动器和控制器错误。
- **1)**磁盘镜像：增设一个完全相同的磁盘驱动器。
- 优点：磁盘驱动器发生故障时切换，仍能正常工作。
- 缺点：磁盘的利用率为**50**％。



磁盘镜像示意图

•2) 磁盘双工

•将两台磁盘驱动器分别接两个磁盘控制器。

• 特点：每个磁盘有自己独立的通道，可同时将数据写入，加块数据读取速度。

数据一致性控制

事务

盘块号一致性检查

链接计数一致性检查

1 事务

- 事务是用于访问和修改各种数据项的一个程序单位。
- 事务也可以被看作是一系列相关读和写操作。

事务记录(Transaction Record):

事务名：用于标识该事务的惟一名字；

·数据项名：它是被修改数据项的惟一名字；

·旧值：修改前数据项的值；

·新值：修改后数据项将具有的值。

恢复算法:可利用以下两个过程实现:

（1） undo〈T_i〉：该过程把所有被事务T_i修改过的数据，恢复为修改前的值。

（2） redo〈T_i〉：该过程能把所有被事务T_i修改过的数据，设置为新值。

如果系统发生故障，系统应对以前所发生的事务进行清理。

2　检查点

检查点(Check Points)的作用

对事务记录表中事务记录的清理工作经常化。

当出现检查点时，利用undo/redo过程实现恢复功能。

新的恢复算法

对最后一个检查点之后的事务记录进行处理。

恢复例程首先查找事务记录表，确定在最近检查点以前开始执行的最后的事务T_i。并利用redo和undo过程对它们进行处理。

如果把所有在事务T_i以后开始执行的事务表示为事务集T， 则新的恢复操作要求：对所有在T中的事务T_K，如果在事务记录表中出现了〈T_K托付〉记录，则执行redo〈T_K〉操作；反之，如果在事务记录表中并未出现〈T_K托付〉记录，则执行undo〈T_K〉操作。

3 并发控制

- 利用互斥锁实现顺序性
- 利用互斥锁和共享锁实现顺序性
  - 共享锁允许多个事务对相应对象执行读操作，而不允许执行写操作。

4 重复数据的数据一致性问题

（1）重复文件的一致性

UNIX类型的文件目录中，每个目录项中含有一个索引结点号，指向该文件的索引结点。对共享文件，该索引结点会在目录中出现多次。

文件名	i 结点
文件 1	17
文件 2	22
文件 3	12
文件 4	84

(*a*)

文件名	i 结点		
文件 1	17	19	40
文件 2	22	72	91
文件 3	12	30	29
文件 4	84	15	66

(*b*)

UNIX类型的目录

（2） 盘块号一致性的检查

盘块号 / 计数器组	0 1	2 3	4 5	6 7	8 9	10 11	12 13	14 15
空闲盘块号计数器组	1 1	0 1	0 1	1 1	1 0	0 1	1 1	0 0
数据盘块号计数器组	0 0	1 0	1 0	0 0	0 1	1 0	0 0	1 1

(*a*) 正常情况

盘块号 / 计数器组	0 1	2 3	4 5	6 7	8 9	10 11	12 13	14 15
空闲盘块号计数器组	1 1	0 1	0 1	1 1	1 0	0 1	1 1	0 0
数据盘块号计数器组	0 0	0 0	1 0	0 0	0 1	1 0	0 0	1 1

(*b*) 丢失了盘块

检查盘块号一致性情况

盘块号 / 计数器组	0 1	2 3	4 5	6 7	8 9	10 11	12 13	14 15
空闲盘块号计数器组	1 1	0 1	2 1	1 1	1 0	0 1	1 1	0 0
数据盘块号计数器组	0 0	1 0	0 0	0 0	0 1	1 0	0 0	1 0

(c) 空闲盘块号重复出现

盘块号 / 计数器组	0 1	2 3	4 5	6 7	8 9	10 11	12 13	14 15
空闲盘块号计数器组	1 1	0 1	1 0	1 1	1 0	0 1	1 1	0 0
数据盘块号计数器组	0 0	1 0	0 2	0 0	0 1	1 0	0 0	1 1

(d) 数据盘块号重复出现

检查盘块号一致性情况

（3） 链接数一致性检查

- **UNIX**中，索引结点在目录中出现的次数，和索引结点中的链接计数应该一致。
- 设置一个计数器表，每个文件建立一个表项，记录该索引结点号的计数值。
- 检查时，从根目录开始查找，当在目录中遇到该索引结点号时，在该计数器表中相应文件的表项上加1。
- 检查完后，将该计数器表中每个表项中的索引结点号计数值与该文件索引结点中的链接计数count值加以比较，如果两者一致，表示是正确的；否则，便是发生了链接数据不一致的错误。
- 当链接计数值与索引结点号计数值不一致时，应将count值置为正确值。